

WPIPE PIPELINES, ENGINEERED

Wpipe: Pipelines that resume, not restart.

Orchestration that scales with you.

+117k

downloads

<50 MB

RAM, total

SQLite

WAL state

Where it hurts today

Three things quietly cost teams time and trust every single week.

State lost on
crash

Retries hand-
rolled

Zero audit trail



wpipe turns these into code you control.

What you gain



Code-first

Pipelines are Python —
versionable, testable,
reviewable.



Resume, don't restart

Every step checkpoints
to SQLite WAL.



Featherweight

<50 MB RAM. No broker,
no daemon.

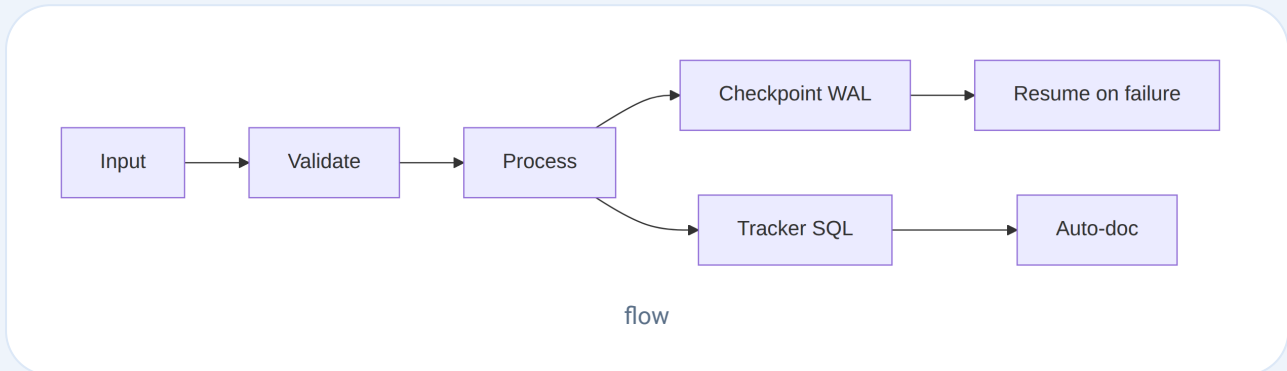
Crash at step 7? Resume from step 4.



Every step persists its state to SQLite WAL — a rerun cost you never pay again.

One pipeline, fully instrumented

State, retries, parallelism and audit — all inside the same run.



wpipe vs. the status quo

Without retries	With wpipe
Fails and restarts everything from zero	Retries with controlled pause
Error at 3 AM, discovered at 9 AM	Error logged with its context
Fear of touching the script	You can iterate without risk

Based on what the engineering community hits daily.

Answers four questions in one run ↘

Did it run, how far, what failed, and can it resume? Yes, yes, exactly, and automatically.

```
python my_pipeline.py
```

SEE IT IN ACTION

Six lines. That's the learning curve.

```
from wpipe import Pipeline, step

@step(name="my_step", retry_count=3, retry_delay=2,
      checkpoint=True)
def my_step(data):
    return {"result": process(data["input"])}

pipe = Pipeline(pipeline_name="my_pipeline", tracking_db="p.db")
pipe.set_steps([my_step])
pipe.run({"input": [...]}))

CHECKPOINTS · RETRIES · TIMEOUT · PARALLEL · TRACKER
```



Code-first

Pipelines are Python —
versionable, testable,
reviewable.



Resume, don't restart

Every step checkpoints
to SQLite WAL.



Featherweight

<50 MB RAM. No broker,
no daemon.

Try wpipe in your next pipeline

Install the library, wrap your step, and watch checkpoints and docs appear by themselves.

```
pip install wpipe
```